

From Algorithms to Articulation

Pirsig, Programming and the Return of Humanist Engineering

Abstract

Programming was once taught as a disciplined descent through levels of description. A student might write in a high-level language yet retain some sense of the assembly instructions, memory operations and machine states beneath it. Contemporary software development increasingly reverses that relation. The programmer states intentions in natural language while hidden systems produce the implementation. This makes the quality of the programmer's concepts, categories and language more consequential, even as direct knowledge of the machine recedes. Robert Pirsig's motorcycle offers a useful starting point for understanding this change, provided the analogy is allowed to fail where it must.

The motorcycle that analysis leaves behind

In *Zen and the Art of Motorcycle Maintenance*, Pirsig performs an exemplary act of technical decomposition. The motorcycle is divided first by components, then by functions. Components divide into assemblies and subassemblies; functions divide into cycles, operations and causal sequences. The analysis is rigorous. It shows how rationality converts a complicated object into a hierarchy that can be named, inspected and repaired.

Yet Pirsig immediately marks the limit of this achievement. The description leaves out the immediate surface through which anyone first encounters a motorcycle. It retains the underlying form, but that form is nearly impossible to understand unless one already knows how a motorcycle works. The decomposition is therefore neither false nor complete. It is a disciplined projection of the motorcycle, useful for certain purposes, while the motorcycle as ridden, heard, balanced, maintained and cared for remains larger than the projection.

Pirsig condenses the point in four short formulations. The machine is a “system of concepts worked out in steel”; it is “primarily a mental phenomenon”; beneath the visible system lies “rationality itself”; and “Quality isn't a method”.¹

These remarks elaborate the motorcycle's materiality while exposing how thoroughly the material machine has already been shaped by distinctions made in thought. A tappet² exists within a designed account of what an engine is, what a valve train does, which motions matter, what counts as tolerance and what counts as failure. Steel receives a form, and the

¹Robert M. Pirsig, *Zen and the Art of Motorcycle Maintenance: An Inquiry into Values* (1974). The linked scan supplied for this essay is available at https://archive.org/details/zenandtheartofmotorcyclenaintenancerobertpirsigm._833_V/page/13/mode/2up.

²A tappet is a small part of an internal-combustion engine's valve mechanism. It transfers the motion of the camshaft to the valve, either directly or through a pushrod and rocker arm, and must maintain a carefully specified clearance.

form carries intentionality guided by a mind. Just as ball bearings are spherical *for a reason*, their form guided by a mind and a purpose, the Earth is spherical *from causes*, without having been designed to be so.³

This is the point at which Pirsig becomes useful for software engineering. Code appears less substantial than steel, but it too is a materialisation of distinctions. The specification of A class, function, table, schema, prompt or ontology is a decision about what exists, what belongs together and what may be ignored.

An engineer more than describes a system in envisioning it.

The old descent through the machine

Traditional programming education presented abstraction as a ladder whose lower rungs remained visible. A programmer might work in Java, C or Python, yet the language was understood as an organised convenience above machine operations. One could, at least in principle, descend:

machine code $\longrightarrow$ assembly $\longrightarrow$ high-level language $\longrightarrow$ algorithm

The direction of teaching usually ran the other way, but the implied justification ran downward. A loop could be explained through jumps and registers; a variable through storage; a method call through stack frames and addresses. The higher language compressed the lower mechanism without entirely concealing it.

This reductive picture had pedagogical power. It made correctness inspectable. The student learned that an apparently fluent program ultimately became precise operations upon a finite machine. *Pseudocode* occupied a valuable position within this arrangement. It abstracted from the incidental syntax of a particular language while preserving sequence, branching, iteration, state and termination. It was a language of processes, neither executable code nor unconstrained prose.

Contentiously, the revised IB Diploma Computer Science Programme course moves away from language-independent pseudocode in favour of explicit Java or Python. That may improve executability and make student work easier to test, but it also risks confusing syntactic correctness with computational understanding. As machines become increasingly capable of generating valid code, the educational priority may need to move in the opposite direction: towards the disciplined formulation, interrogation and validation of the idea before its implementation. Pseudocode mattered because it occupied that intermediate space. It required more precision than ordinary language while refusing to let the grammar of a particular programming language become the thought itself.⁴

The difficulty is that contemporary programming is no longer simply another ascent to a still higher-level language. In AI-assisted development, the programmer may specify a desired behaviour while lacking any realistic capacity to descend through the generated

³This distinction follows Daniel Dennett's contrast between explaining an artefact from the *design stance* and explaining a natural object through its causal history. See Daniel C. Dennett, *The Intentional Stance* (Cambridge, MA: MIT Press, 1987), and *Darwin's Dangerous Idea* (London: Penguin, 1995).

⁴The revised IB Diploma Programme Computer Science course was launched in 2025 for first assessment in 2027. It replaces the former language-independent pseudocode used in assessment with programming in Java or Python.

implementation. Libraries already made complete descent difficult; generated code and model-mediated systems can make it practically impossible. The abstraction ladder has become a curtain.

This does not mean that machine knowledge has become worthless. It means that the old justification of programming solely through reduction is no longer sufficient. The programmer must increasingly judge systems whose internal operations cannot be fully surveyed. Verification, provenance, architecture and purpose therefore become more important at the same moment that direct authorship becomes less complete.

Where the motorcycle analogy begins to fail

Pirsig's mechanic can remove a cover, measure a clearance and follow a causal chain through the engine. The machine resists, but it is locally inspectable. Even when its hierarchy does not capture the lived motorcycle, the relation between component and function remains comparatively stable.

Modern software is less obliging. Its behaviour may emerge from distributed services, learned parameters, external databases, shifting dependencies and retrieved documents. No single engineer necessarily possesses the whole causal account. The system can continue to function while no one can narrate it from instruction to outcome.

At this point the motorcycle metaphor begins to flounder. The more useful comparison may be the body. An organ can be isolated anatomically and assigned a function, yet its actual operation depends upon circulatory, hormonal, neural, microbial and environmental relations. The heart is a pump, but that description is radically insufficient as an account of circulation, exertion, fear or life. Form and function remain distinguishable, though neither is intelligible in isolation⁵.

Software systems increasingly resemble this kind of organisation. Their parts are not merely assembled; they are situated in networks of dependency, information and interpretation. A database field changes meaning according to who entered it, why it was collected, which categories were available and how later systems retrieve it. A model output acquires significance only through the human institution that acts upon it. The relevant unit is no longer always the object or algorithm. It may be the relation, the context, the claim or the evidential chain.

The body analogy should also be handled gently. Software is designed in ways bodies are not, and biological holism can become an excuse for vagueness. Its value is narrower: it reminds us that decomposition does not guarantee explanation. A list of organs does not amount to physiology, and a list of functions does not amount to a person. Likewise, a repository does not amount to the working meaning of a software system.

From algorithmic language to relational language

Classical pseudocode asks a procedural question:

⁵The analogy with bodily organs points beyond straightforward mechanical decomposition. Denis Noble argues that the organism is not merely a vehicle constructed and directed by its genes: causation operates across molecular, cellular, organ and organismal levels. The score does not contain the performance, nor can the music be reduced to its individual notes. See Denis Noble, *The Music of Life: Biology Beyond Genes* (Oxford: Oxford University Press, 2006).

What sequence of operations transforms this input into the required output?

That question remains indispensable. Yet databases, knowledge graphs and retrieval-augmented generation introduce another family of questions:

What entities are present? How are they related? Which claims are supported?

What context limits the inference? Which evidence should be retrieved?

These questions require a language of ideas and relations as well as a language of processes. A school-level pseudocode might therefore contain both procedural and declarative forms:

```
ENTITY Tappet
```

```
ENTITY Engine
```

```
Tappet PART-OF Engine
```

```
CLAIM ExcessiveClearance CAUSES TappingNoise
```

```
SUPPORTED-BY ServiceManual.Section4
```

```
APPLIES-WHEN Engine = Cold
```

```
FIND probable_cause
```

```
WHERE symptom = TappingNoise
```

```
RETURN claim, evidence, conditions
```

This is not a replacement for algorithms. Retrieval still requires ranking; databases still require execution plans; graph queries still become machine operations. The educational shift is that the student must learn to formalise not only *what happens next*, but also *what is being claimed*. The central intellectual acts become naming, relating, constraining and justifying.

This gives renewed importance to ontology, semantics and even etymology. Names are not cosmetic labels placed on completed structures. They carry inherited distinctions. A term may gather meanings from law, medicine, commerce or ordinary speech before entering a database schema. Once formalised, those meanings become selectable fields, categories and edges. The machine then reproduces the distinction at scale.

The older engineer could imagine that values entered only after the technical work was finished. In a knowledge graph or RAG system, values are already present in the ontology: in what is admitted as an entity, what is treated as evidence, which relations are searchable and whose language becomes canonical.

Humanist engineering

This is the strongest case for humanist engineering. It does not ask the engineer to become less precise or to exchange technical competence for cultural commentary. It asks for precision at the level where contemporary systems are actually being designed.

A generated program may be syntactically competent while embodying a poor account of the world. A retrieval system may answer fluently while collapsing testimony, policy, measurement and opinion into undifferentiated text. A knowledge graph may be logically consistent while reproducing crude or historically loaded categories. These are failures of engineering because they are failures in the concepts worked out through the system.

The humanities become relevant here as disciplines of interpretation. History shows how categories were formed and whom they served. Philosophy distinguishes evidence from assertion, identity from classification and description from value. Linguistics examines how meaning shifts with context. Literature trains attention upon voice, ambiguity, motive and what a formal account leaves unsaid. Ethics asks which purposes deserve implementation.

Pirsig's Quality belongs in this space. It is not an ornamental preference added after correctness. It is the cultivated capacity to recognise that a technically adequate decomposition may still miss the object it claims to describe. The rainbow can be separated into wavelengths, but the account of wavelength does not exhaust colour. The motorcycle can be divided into components and functions, but the resulting hierarchy does not ride. The program can be reduced to instructions, but the instructions do not explain what the program means within a human practice.

Older engineering often treated such remainder as softness, subjectivity or noise. Contemporary engineering can no longer afford that dismissal. When machines increasingly generate the algorithms, engineers assume greater responsibility for the language of intention, relation and evidence surrounding them.

A revised educational sequence

Computer science education should therefore resist two symmetrical errors. It should not abandon executable code for vague prompting. Nor should it respond to AI by retreating into syntax as though manual code production were still the whole discipline.

A more coherent sequence would move repeatedly among four levels:

- **Algorithm:** the sequence of operations and changes of state;
- **Implementation:** the executable language, runtime and machine constraints;
- **Representation:** the entities, relations, schemas and evidence structures through which the problem is expressed;
- **Quality:** the judgement of whether the representation and resulting system are apt, humane and worth sustaining.

Pseudocode remains central, but its scope should widen. Alongside loops, branches and procedures, students should write controlled relational statements, queries, provenance rules and explicit conditions of applicability. They should learn to move from natural-language intention to a formal intermediate representation, then to executable code, and finally back to a human evaluation of the result.

The decisive future skill may not be the ability to produce every instruction personally. It may be the ability to articulate an intention without surrendering it: to name the right entities, preserve relevant distinctions, specify relations, demand evidence and recognise when an apparently functioning system has misunderstood its object.

That is not a retreat from engineering into the humanities. It is an acknowledgement that engineering has reached the point at which its most consequential materials are concepts.

Source note. The discussion of Pirsig's component and functional decomposition follows the motorcycle hierarchy passage in *Zen and the Art of Motorcycle Maintenance*. A searchable transcription is also available at <https://terebess.hu/english/motor2.html>. All extended wording above is paraphrased; only the four brief phrases explicitly marked as quotations reproduce Pirsig's text.